

Explicação do Código Java: ConsistentHashing

1. O código implementa hashing consistente para distribuir chaves (ex.: utilizadores) entre servidores com mínima redistribuição ao adicionar/remover nós.
2. Usa TreeMap como anel ordenado de hashes, onde cada posição aponta para um servidor.
3. addNode() cria vários nós virtuais por servidor, melhorando equilíbrio de carga e reduzindo hotspots.
4. removeNode() elimina do anel todos os hashes virtuais associados ao servidor removido.
5. getNode() calcula hash da chave e procura ceilingEntry(): o primeiro nó com hash maior ou igual.
6. Se não existir posição superior, volta ao primeiro elemento do mapa, simulando o anel circular.
7. Conceitos Java usados: coleções (TreeMap), generics, Map.Entry, métodos de String e hashCode().
8. Complexidade típica: busca $O(\log n)$; inserção/remoção $O(v \log n)$, onde v = nós virtuais por servidor.
9. Decisão importante: nós virtuais aumentam balanceamento, mas consomem mais memória e operações.
10. Limitação: hashCode() pode colidir e distribuir pior que funções hash criptográficas/especializadas.